

天津大学

《智能搜索与问答》实验报告



Lab5 对话状态追踪系统的实现

学 号 3021244141

姓 名 于梦洁

学 院 智能与计算学部

专 业 计算机科学与技术

年 级 2021 级

班 级 2 班

2024 年 4 月 27 日

目录

一、 数据准备与处理	3
1.1 数据集	3
1.2 数据预处理	5
二、 模型与算法介绍	5
2.1 任务介绍	5
2.2 模型介绍	6
2.3 评价指标	7
三、 实验流程	8
3.1 使用 conda 创建虚拟环境	8
3.2 对模型进行训练	9
3.3 对模型进行测试	11
四、 实验结果	11
五、 结果分析	13
六、 心得体会	13

一、 数据准备与处理

1.1 数据集

本实验使用的数据集是 RiSAWOZ，它是一个具有丰富语义信息标注的大规模中文多领域任务型对话数据集，它包含 1.12 万个已标注的人-人多轮对话，总对话轮数超过 15 万轮，覆盖 12 个领域。其中，单领域和多领域的对话分别占数据集的 65%和 35%。它是目前最大的的中文任务型对话数据集。

User goal:

你刚到苏州旅游，想咨询些信息。首先，你想在姑苏区找一家价格便宜的酒店，询问酒店的房费和地址。其次，你希望客服推荐这附近最热门的苏州园林，并询问它的地址和特点。然后，你还想在附近找一家有名的江浙菜餐厅，并询问他们家的推荐菜。最后你感谢客服的帮助并说再见。

You have just arrived in Suzhou for a tour and want to ask for some information. First of all, you want to find a cheap hotel in Gusu District, and ask about the hotel's room rate and address. Secondly, you would like the customer service to recommend the most popular Suzhou Garden nearby, and ask about its address and characteristics. Then, you want to find a famous restaurant nearby which serves Jiangsu and Zhejiang cuisine and ask about recommended dishes of the restaurant. Finally, you thank the customer service for his help and say goodbye.

Dialogue domains: [Hotel, Attraction, Restaurant]

你好。我是来苏州旅游的，我能咨询一些姑苏区的酒店信息吗？

Hello. I'm traveling in Suzhou. Can I ask for some information about hotels in Gusu District?

User actions: General: Greeting, Hotel: Inform (area=Gusu District)

Dialogue state: Hotel: area=Gusu District

您好，很高兴为您服务。那里酒店很多。请问您想找什么价位的酒店呢？

Hello, glad to serve you. There are many hotels in that area. What price range do you prefer?

System actions: General: Greeting, Hotel: Request (pricerange=?)

找那边价格便宜点的吧。房费多少钱？

Cheap one there please. How much is the room rate?

Rewriting utterances for ellipsis and coreference resolution

User actions: Hotel: Inform (pricerange=cheap), Hotel: Request (room rate=?)

Dialogue state: Hotel: area=Gusu District, pricerange=cheap

图 1. Dialogue example

```

"旅游景点": {
  "名称": [ ...
  ],
  "区域": [
    "工业园区",
    "常熟",
    "姑苏区",
    "昆山",
    "高新区",
    "吴中区",
    "吴江"
  ],
  "景点类型": [ ...
  ],
  "最适合人群": [ ...
  ],
  "消费": [ ...
  ],
  "是否地铁直达": [ ...
  ],
  "门票价格": [ ...
  ],
  "电话号码": [ ...
  ],
  "地址": [ ...
  ],
  "评分": [ ...
  ],
  "开放时间": [ ...
  ],
  "特点": [ ...
  ]
},

```

图 2. Ontology

Dialogues	10,000
Turns	134,580
Domains	12
Slots	159
Values	4061

图 3. Dialogue 内容分布

1.2 数据预处理

```
data_files = {}
if data_args.train_file is not None:
    data_files["train"] = data_args.train_file
    extension = data_args.train_file.split(".")[1]
if data_args.validation_file is not None:
    data_files["validation"] = data_args.validation_file
    extension = data_args.validation_file.split(".")[1]
if data_args.test_file is not None:
    data_files["test"] = data_args.test_file
    extension = data_args.test_file.split(".")[1]
raw_datasets = load_dataset(
    extension,
    data_files=data_files,
    cache_dir=model_args.cache_dir,
    use_auth_token=True if model_args.use_auth_token else None,
)
```

图 4. 数据加载代码

```
def preprocess_function(examples):
    inputs, targets = [], []
    for i in range(len(examples[text_column])):
        if examples[text_column][i] and examples[summary_column][i]:
            inputs.append(examples[text_column][i])
            targets.append(examples[summary_column][i])

    inputs = [prefix + inp for inp in inputs]
    model_inputs = tokenizer(inputs, max_length=data_args.max_source_length, padding=padding, truncation=True)

    with tokenizer.as_target_tokenizer():
        labels = tokenizer(targets, max_length=max_target_length, padding=padding, truncation=True)

    if padding == "max_length" and data_args.ignore_pad_token_for_loss:
        labels["input_ids"] = [
            [(l if l != tokenizer.pad_token_id else -100) for l in label] for label in labels["input_ids"]
        ]

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

图 5. 数据预处理代码

二、 模型与算法介绍

2.1 任务介绍

对话状态追踪（DST, Dialogue State Tracking）是自然语言处理中的一个重要技术，它主要用于识别用户当前所处的对话状态，并将其与历史记录联系起来，以便基于用户的真实意图进行有效、准确的回复。DST 在 pipeline 对话系统中是一个至关重要的模块，它上接自然语言理解（NLU），下接对话策略（Policy）。其中，对话状态是人机对话过程中，用户目标（Goal）的达成状态，一般用 (domain, intent, slot) 标识。

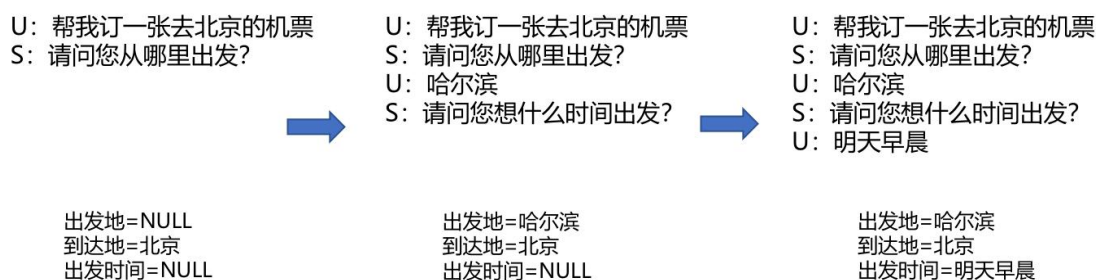


图 6. 对话状态示例

对于对话追踪系统，他的对话角色为用户和系统，输出格式为领域-槽名-槽值，以逗号分隔。

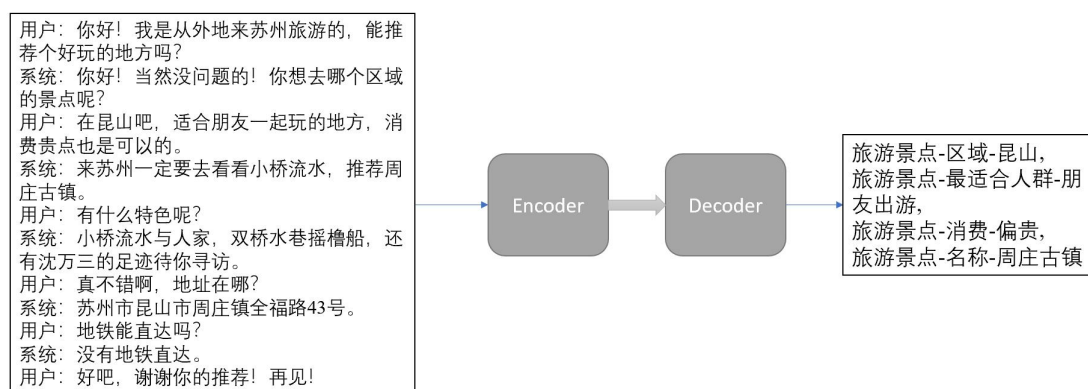


图 7. 系统框架

2.2 模型介绍

对于本实验使用的模型是 Randeng-T5-77M，该模型的架构主要基于谷歌提出的 T5（Text-to-Text Transfer Transformer）模型，是一种基于 Transformer 的 encoder-decoder 架构。

在 T5 模型中，encoder 和 decoder 都使用了 Transformer 的堆叠结构，即多个自注意力机制和前馈神经网络的组合。这种结构使得模型能够处理任意长度的输入序列，并通过自注意力机制捕获输入序列中的依赖关系。

对于 Randeng-T5-77M 模型，其参数规模约为 77M（百万），这意味着它在处理 NLP 任务时具有一定的效率和准确性。由于该模型是针对中文语言环境和任务进行优化和训练的，因此它采用了中文分词器（如 sentence piece）来处理中文文本。

在架构上，Randeng-T5-77M 模型遵循 T5 模型的通用设计原则，即将所有 NLP 任务都转化为 text-to-text 的格式。这意味着模型的输入和输出都是文本序列，通过 encoder 将输入文本转化为一个向量表示，然后通过 decoder 将这个向量表示转化为输出文本。

在训练过程中，Randeng-T5-77M 模型采用了多任务学习的方法，通过同时训练多个 NLP 任务来增强模型的泛化能力。这些任务包括情感分析、新闻分类、文本分类、意图识别、自然语言推理、多项选择、指代消解、抽取式阅读理解等。通过多任务学习，Randeng-T5-77M 模型能够学习到这些任务之间的共性和差异，从而提高其在各个任务上的性能。


```

config = AutoConfig.from_pretrained(
    model_args.config_name if model_args.config_name else model_args.model_name_or_path,
    cache_dir=model_args.cache_dir,
    revision=model_args.model_revision,
    use_auth_token=True if model_args.use_auth_token else None,
)
tokenizer = AutoTokenizer.from_pretrained(
    model_args.tokenizer_name if model_args.tokenizer_name else model_args.model_name_or_path,
    cache_dir=model_args.cache_dir,
    use_fast=model_args.use_fast_tokenizer,
    revision=model_args.model_revision,
    use_auth_token=True if model_args.use_auth_token else None,
)
model = AutoModelForSeq2SeqLM.from_pretrained(
    model_args.model_name_or_path,
    from_tf=bool(".ckpt" in model_args.model_name_or_path),
    config=config,
    cache_dir=model_args.cache_dir,
    revision=model_args.model_revision,
    use_auth_token=True if model_args.use_auth_token else None,
)

```

图 8. 加载模型代码

2.3 评价指标

Joint Goal Accuracy（联合目标准确率）是对话状态追踪（DST）任务中的一个重要评价指标。具体来说，对于每轮对话，该指标将预测的对话状态和真实的对话状态进行比较。只有当对话状态中所有的（domain, slot, value）预测正确时，才认为对话状态预测正确。因此，Joint Goal Accuracy 能够全面衡量 DST 模型在预测对话状态时的准确性和性能。

旅游景点-区域：吴中区 旅游景点-最适合人群：朋友出游 旅游景点-消费：中等 旅游景点-名称：天平山景区 餐厅-价位：中等 餐厅-菜系：火锅 餐厅-名称：四川香天下火锅	旅游景点-区域：吴中区 旅游景点-最适合人群：朋友出游 旅游景点-消费：中等 旅游景点-名称：天平山景区 餐厅-价位：偏贵 餐厅-菜系：火锅 餐厅-名称：四川香天下火锅
Predicted slot values	Ground truth
✗	

图 9. 测评示例

```

predict_results = trainer.predict(
    predict_dataset, metric_key_prefix="predict", max_length=max_length, num_beams=num_beams
)
metrics = predict_results.metrics
max_predict_samples = (
    data_args.max_predict_samples if data_args.max_predict_samples is not None else len(predict_dataset)
)
metrics["predict_samples"] = min(max_predict_samples, len(predict_dataset))

trainer.log_metrics("predict", metrics)
trainer.save_metrics("predict", metrics)

if trainer.is_world_process_zero():
    if training_args.predict_with_generate:
        predictions = tokenizer.batch_decode(
            predict_results.predictions, skip_special_tokens=True, clean_up_tokenization_spaces=True
        )
        predictions = [pred.strip() for pred in predictions]
        output_prediction_file = os.path.join(training_args.output_dir, "generated_predictions.txt")
        with open(output_prediction_file, "w") as writer:
            writer.write("\n".join(predictions))

```

图 9. 评测代码

总的来说，Randeng-T5-77M 模型是一种基于 T5 架构的中文多任务模型，它采用了 Transformer 的 encoder-decoder 架构，并通过多任务学习来增强模型的泛化能力。这种架构使得 Randeng-T5-77M 模型在处理中文 NLP 任务时具有较高的效率和准确性。

三、 实验流程

3.1 使用 conda 创建虚拟环境

使用命令 “conda create -n lab5 python=3.8” 创建需要的环境，然后使用命令 “conda activate lab3” 激活环境。

```

(base) root@autodl-container-4fcell19e52-1d7b1e9d: /autodl-tmp/dst_code# conda create -n lab5 python=3.8
Collecting package metadata (current_repodata.json): done
Solving environment: done
## Package Plan ##

  environment location: /root/miniconda3/envs/lab5

added / updated specs:
- python=3.8

The following packages will be downloaded:

package | build | size | url
-----|-----|-----|-----
_libgcc_mutex-0.1 | conda_forge | 3 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
_openmp_mutex-4.5 | 2_gnu | 23 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
bzip2-1.0.8 | hd590300_5 | 243 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
ca-certificates-2024.2.2 | hbcca054_0 | 152 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
ld_impl_linux-64-2.40 | h55db66e_0 | 697 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
libffi-3.4.2 | h7f98852_5 | 57 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
libgcc-ng-13.2.0 | h77fa898_7 | 758 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
libgomp-13.2.0 | h77fa898_7 | 412 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
libns1-2.0.1 | hd590300_0 | 33 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
libsqlite-3.45.3 | h2797004_0 | 840 KB | https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge

```

图 10. 创建名为 lab5 的虚拟环境

```

(base) root@autodl-container-4fcell19e52-1d7b1e9d: /autodl-tmp/dst_code# conda activate lab5
(lab5) root@autodl-container-4fcell19e52-1d7b1e9d: /autodl-tmp/dst_code# pip install -r requirements.txt

```

图 11. 激活虚拟环境

运行命令“pip install -r requirements.txt”下载所需要的环境以及数据。

```
(lab5) root@autodl-container-4fcell19e52-1d7b1e9d: /autodl-tmp/dst_code# pip install -r requirements.txt
Looking in indexes: http://mirrors.aliyun.com/pypi/simple
Collecting datasets (from -r requirements.txt (line 1))
  Downloading http://mirrors.aliyun.com/pypi/packages/9f/8a/3922b6d4a8fb40db454abd5d66b28215b047563564f044de693643d5d07f/datasets-2.19.1-py3-non
e-any.whl (542 kB)
-----542.0/542.0 kB 2.1 MB/s eta 0:00:00
Collecting filelock (from -r requirements.txt (line 2))
  Downloading http://mirrors.aliyun.com/pypi/packages/41/24/0b023b6537dfc9bae2c779353998e3e99ac7dfff4222fc6126650e93c3f3/filelock-3.14.0-py3-non
e-any.whl (12 kB)
Collecting nltk (from -r requirements.txt (line 3))
  Downloading http://mirrors.aliyun.com/pypi/packages/a6/0a/0d20d2c0f16be91b9fa32a77b76c60f9baf6eba419e5ef5deca17af9c582/nltk-3.8.1-py3-none-any
.whl (1.5 MB)
-----1.5/1.5 MB 2.0 MB/s eta 0:00:00
Collecting numpy (from -r requirements.txt (line 4))
  Downloading http://mirrors.aliyun.com/pypi/packages/98/5d/5738903efe0ecb73e51eb44feafba32bda2081263d40c5043568f60faf/numpy-1.24.4-cp38-cp38-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (17.3 MB)
-----17.3/17.3 MB 1.9 MB/s eta 0:00:00
Collecting transformers==4.21.2 (from -r requirements.txt (line 5))
  Downloading http://mirrors.aliyun.com/pypi/packages/70/2f/fe0e399c50b0f2b695ed1d00dc084f06888d20290fc74224f9dd12f11fc5/transformers-4.21.2-py3
-none-any.whl (4.7 MB)
-----4.7/4.7 MB 1.8 MB/s eta 0:00:00
Collecting huggingface-hub<1.0,>=0.1.0 (from transformers==4.21.2->-r requirements.txt (line 5))
  Downloading http://mirrors.aliyun.com/pypi/packages/21/2b/516f82c5ba9eb184b24c11976be2ad5e80fb7fe6b2796c887087144445e/huggingface_hub-0.23.0-
py3-none-any.whl (401 kB)
-----401.2/401.2 kB 1.7 MB/s eta 0:00:00
Collecting packaging>=20.0 (from transformers==4.21.2->-r requirements.txt (line 5))
  Downloading http://mirrors.aliyun.com/pypi/packages/49/df/1fceb2f8900f8639e278b056416d49134fb8d84c5942ffaa01ad34782422/packaging-24.0-py3-none
-any.whl (53 kB)
-----53.5/53.5 kB 2.3 MB/s eta 0:00:00
Collecting pyyaml>=5.1 (from transformers==4.21.2->-r requirements.txt (line 5))
  Downloading http://mirrors.aliyun.com/pypi/packages/c8/6b/6600ac24725c7388255b2f5add93f91e58a5d7efaf4af244fdbcc11a541b/PyYAML-6.0.1-cp38-cp38-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (736 kB)
-----736.6/736.6 kB 2.0 MB/s eta 0:00:00
Collecting regex!=2019.12.17 (from transformers==4.21.2->-r requirements.txt (line 5))
  Downloading http://mirrors.aliyun.com/pypi/packages/ec/a1/2542d273818b140a28d26cec2956a0ce8c8ade25de1ed4f515c08593d4ea/regex-2024.5.15-cp38-cp
38-
```

图 12. 下载需要的环境和数据集

3.2 对模型进行训练

运行命令行“sh train.sh”对模型进行训练。

```
(lab5) root@autodl-container-4fcell19e52-1d7b1e9d: /autodl-tmp/dst_code# sh train.sh
/root/miniconda3/envs/lab5/lib/python3.8/site-packages/datasets/load.py:2547: FutureWarning: 'use_auth_token' was deprecated in favor of 'token' in version 2.14.0 and will be r
emoved in 3.0.0.
You can remove this warning by passing 'token=<use_auth_token>' instead.
  warnings.warn(
Using custom data configuration default-45b7308122cad10b
05/17/2024 15:26:10 - INFO - datasets.builder - Using custom data configuration default-45b7308122cad10b
Loading Dataset Infos from /root/miniconda3/envs/lab5/lib/python3.8/site-packages/datasets/packaged_modules/json
05/17/2024 15:26:10 - INFO - datasets.info - Loading Dataset Infos from /root/miniconda3/envs/lab5/lib/python3.8/site-packages/datasets/packaged_modules/json
Generating dataset json (/root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecf34c3700d143b31835ec4235bf10109f7)
05/17/2024 15:26:10 - INFO - datasets.builder - Generating dataset json (/root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecf
f34c3700d143b31835ec4235bf10109f7)
Downloading and preparing dataset json/default to /root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecf34c3700d143b31835ec423
5bf10109f7...
05/17/2024 15:26:10 - INFO - datasets.builder - Downloading and preparing dataset json/default to /root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9
508a2a2067ab02cd118834ecf34c3700d143b31835ec4235bf10109f7...
Downloading took 0.0 min
05/17/2024 15:26:10 - INFO - datasets.download.download_manager - Downloading took 0.0 min
Checksum Computation took 0.0 min
05/17/2024 15:26:10 - INFO - datasets.download.download_manager - Checksum Computation took 0.0 min
Generating train split
05/17/2024 15:26:10 - INFO - datasets.builder - Generating train split
Generating train split: 67290 examples [00:00, 451817.00 examples/s]
Generating validation split
05/17/2024 15:26:10 - INFO - datasets.builder - Generating validation split
Generating validation split: 4058 examples [00:00, 451435.84 examples/s]
Generating test split
05/17/2024 15:26:10 - INFO - datasets.builder - Generating test split
Generating test split: 4643 examples [00:00, 421254.05 examples/s]
Unable to verify splits sizes.
05/17/2024 15:26:10 - INFO - datasets.utils.info_utils - Unable to verify splits sizes.
Dataset json downloaded and prepared to /root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecf34c3700d143b31835ec4235bf10109f7
. Subsequent calls will reuse this data.
05/17/2024 15:26:10 - INFO - datasets.builder - Dataset json downloaded and prepared to /root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067
ab02cd118834ecf34c3700d143b31835ec4235bf10109f7. Subsequent calls will reuse this data.
```

图 13. 训练示例图（1）

```

[INFO]configuration_utils.py:679] 2024-05-17 15:26:10,261 >> loading configuration file ./pretrained_model/Randeng-T5-77M/config.json
[INFO]configuration_utils.py:730] 2024-05-17 15:26:10,262 >> Model config MT5Config {
  "_name_or_path": "./pretrained_model/Randeng-T5-77M",
  "architectures": [
    "MT5ForConditionalGeneration"
  ],
  "d_ff": 1024,
  "d_kv": 64,
  "d_model": 512,
  "decoder_start_token_id": 0,
  "dense_act_fn": "gelu_new",
  "dropout_rate": 0.1,
  "eos_token_id": 1,
  "feed_forward_proj": "gated-gelu",
  "initializer_factor": 1.0,
  "is_encoder_decoder": true,
  "is_gated_act": true,
  "layer_norm_epsilon": 1e-06,
  "model_type": "mt5",
  "num_decoder_layers": 8,
  "num_heads": 6,
  "num_layers": 8,
  "pad_token_id": 0,
  "relative_attention_max_distance": 128,
  "relative_attention_num_buckets": 32,
  "tie_word_embeddings": false,
  "tokenizer_class": "T5Tokenizer",
  "torch_dtype": "float16",
  "transformers_version": "4.21.2",
  "use_cache": true,
  "vocab_size": 32596
}

[INFO]tokenization_utils_base.py:1723] 2024-05-17 15:26:10,263 >> Didn't find file ./pretrained_model/Randeng-T5-77M/added_tokens.json. We won't load it.
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:26:10,263 >> loading file ./pretrained_model/Randeng-T5-77M/spiece.model
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:26:10,263 >> loading file ./pretrained_model/Randeng-T5-77M/tokenizer.json
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:26:10,263 >> loading file None
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:26:10,263 >> loading file ./pretrained_model/Randeng-T5-77M/special_tokens_map.json
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:26:10,263 >> loading file ./pretrained_model/Randeng-T5-77M/tokenizer_config.json
[INFO]modeling_utils.py:2039] 2024-05-17 15:26:10,587 >> loading weights file ./pretrained_model/Randeng-T5-77M/pytorch_model.bin
[INFO]modeling_utils.py:2435] 2024-05-17 15:26:11,536 >> All model checkpoint weights were used when initializing MT5ForConditionalGeneration.

[INFO]trainer.py:1605] 2024-05-17 15:26:22,975 >> ***** Running training *****
[INFO]trainer.py:1606] 2024-05-17 15:26:22,975 >> Num examples = 67290
[INFO]trainer.py:1607] 2024-05-17 15:26:22,975 >> Num Epochs = 1
[INFO]trainer.py:1608] 2024-05-17 15:26:22,975 >> Instantaneous batch size per device = 4
[INFO]trainer.py:1609] 2024-05-17 15:26:22,975 >> Total train batch size (w. parallel, distributed & accumulation) = 4
[INFO]trainer.py:1610] 2024-05-17 15:26:22,975 >> Gradient Accumulation steps = 1
[INFO]trainer.py:1611] 2024-05-17 15:26:22,975 >> Total optimization steps = 16823
{'loss': 17.9223, 'learning_rate': 2.9708853238265006e-06, 'epoch': 0.0}
{'loss': 10.0649, 'learning_rate': 5.941770647653001e-06, 'epoch': 0.01}
{'loss': 6.7016, 'learning_rate': 8.912655971479501e-06, 'epoch': 0.01}
{'loss': 5.6506, 'learning_rate': 1.1883541296306002e-05, 'epoch': 0.01}
{'loss': 4.0921, 'learning_rate': 1.4854426619132503e-05, 'epoch': 0.01}
{'loss': 3.0883, 'learning_rate': 1.7825311942969002e-05, 'epoch': 0.02}
{'loss': 2.5363, 'learning_rate': 2.0796197266785503e-05, 'epoch': 0.02}
{'loss': 2.208, 'learning_rate': 2.3767082590612004e-05, 'epoch': 0.02}
{'loss': 1.7919, 'learning_rate': 2.6737967914438505e-05, 'epoch': 0.03}
{'loss': 1.6368, 'learning_rate': 2.9708853238265006e-05, 'epoch': 0.03}
3% [ 500/16823 [00:39<20:52, 13.03it/s]

[INFO]trainer.py:2640] 2024-05-17 15:27:02,106 >> Saving model checkpoint to ./output/checkpoint-500
[INFO]configuration_utils.py:451] 2024-05-17 15:27:02,107 >> Configuration saved in ./output/checkpoint-500/config.json
[INFO]modeling_utils.py:1566] 2024-05-17 15:27:02,491 >> Model weights saved in ./output/checkpoint-500/pytorch_model.bin
[INFO]tokenization_utils_base.py:2145] 2024-05-17 15:27:02,491 >> tokenizer config file saved in ./output/checkpoint-500/tokenizer_config.json
[INFO]tokenization_utils_base.py:2152] 2024-05-17 15:27:02,492 >> Special tokens file saved in ./output/checkpoint-500/special_tokens_map.json
{'loss': 1.5197, 'learning_rate': 3.2679728562091506e-05, 'epoch': 0.03}
{'loss': 1.3516, 'learning_rate': 3.5850623885918004e-05, 'epoch': 0.04}
{'loss': 1.2636, 'learning_rate': 3.862159392974451e-05, 'epoch': 0.04}
{'loss': 1.0852, 'learning_rate': 4.1592394533571006e-05, 'epoch': 0.04}
{'loss': 1.0495, 'learning_rate': 4.45632798573975e-05, 'epoch': 0.04}
{'loss': 1.0115, 'learning_rate': 4.753416518122401e-05, 'epoch': 0.05}
{'loss': 0.9491, 'learning_rate': 5.050505050505051e-05, 'epoch': 0.05}
{'loss': 0.8935, 'learning_rate': 5.347593582887701e-05, 'epoch': 0.05}
{'loss': 0.778, 'learning_rate': 5.6446821152703514e-05, 'epoch': 0.06}
{'loss': 0.7851, 'learning_rate': 5.941770647653001e-05, 'epoch': 0.06}
6% [ 1000/16823 [01:20<20:40, 12.76it/s]

[INFO]trainer.py:2640] 2024-05-17 15:27:43,899 >> Saving model checkpoint to ./output/checkpoint-1000

```

图 14.训练示例图（2）

```

{'train_runtime': 1387.0665, 'train_samples_per_second': 48.512, 'train_steps_per_second': 12.128, 'train_loss': 0.2866433160041279, 'epoch': 1.0}
100% [ 16823/16823 [23:07<00:00, 12.13it/s]

[INFO]trainer.py:2640] 2024-05-17 15:49:30,042 >> Saving model checkpoint to ./output
[INFO]configuration_utils.py:451] 2024-05-17 15:49:30,043 >> Configuration saved in ./output/config.json
[INFO]modeling_utils.py:1566] 2024-05-17 15:49:30,429 >> Model weights saved in ./output/pytorch_model.bin
[INFO]tokenization_utils_base.py:2145] 2024-05-17 15:49:30,430 >> tokenizer config file saved in ./output/tokenizer_config.json
[INFO]tokenization_utils_base.py:2152] 2024-05-17 15:49:30,430 >> Special tokens file saved in ./output/special_tokens_map.json
***** train metrics *****
epoch                = 1.0
train_loss           = 0.2866
train_runtime        = 0:23:07.06
train_samples        = 67290
train_samples_per_second = 48.512
train_steps_per_second  = 12.128

```

图 15.训练示例图（3）

3.3 对模型进行测试

运行命令行“sh test.sh”对模型进行测试。

```
(lab5) root@autodl-container-4fc0119e52-1d7b1e9d: /autodl-tmp/dst_code# sh test.sh
/root/miniconda3/envs/lab5/lib/python3.8/site-packages/datasets/load.py:2547: FutureWarning: 'use_auth_token' was deprecated in favor of 'token' in version 2.14.0 and will be removed in 3.0.0.
You can remove this warning by passing 'token=(use_auth_token)' instead.
  warnings.warn(
Using custom data configuration default-45b7308122cad10b
05/17/2024 15:49:43 - INFO - datasets.builder - Using custom data configuration default-45b7308122cad10b
Loading Dataset Infos from /root/miniconda3/envs/lab5/lib/python3.8/site-packages/datasets/packaged_modules/json
05/17/2024 15:49:43 - INFO - datasets.info - Loading Dataset Infos from /root/miniconda3/envs/lab5/lib/python3.8/site-packages/datasets/packaged_modules/json
Overwrite dataset info from restored data version if exists.
05/17/2024 15:49:43 - INFO - datasets.builder - Overwrite dataset info from restored data version if exists.
Loading Dataset info from ./cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecef34c3700d143b31835ec4235bf10109f7
05/17/2024 15:49:43 - INFO - datasets.info - Loading Dataset info from ./cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecef34c3700d143b31835ec4235bf10109f7
Found cached dataset json (/root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecef34c3700d143b31835ec4235bf10109f7)
05/17/2024 15:49:43 - INFO - datasets.builder - Found cached dataset json (/root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecef34c3700d143b31835ec4235bf10109f7)
Loading Dataset info from /root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecef34c3700d143b31835ec4235bf10109f7
05/17/2024 15:49:43 - INFO - datasets.info - Loading Dataset info from /root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecef34c3700d143b31835ec4235bf10109f7
[INFO]configuration_utils.py:679] 2024-05-17 15:49:48,653 >> loading configuration file ./output/config.json
[INFO]configuration_utils.py:730] 2024-05-17 15:49:48,655 >> Model config MT5ForConditionalGeneration {
  "name_or_path": "./output",
  "architectures": [
    "MT5ForConditionalGeneration"
  ],
  "d_ff": 1024,
  "d_kv": 64,
  "d_model": 512,
  "decoder_start_token_id": 0,
  "dense_act_fn": "gelu_new",
  "dropout_rate": 0.1,
  "eos_token_id": 1,
  "feed_forward_proj": "gated-gelu",
  "initializer_factor": 1.0,
  "is_encoder_decoder": true,
  "is_gated_act": true,
  "layer_norm_epsilon": 1e-06,
  "model_type": "mt5",
  "num_decoder_layers": 8,
  "num_heads": 6,
  "pad_token_id": 0,
  "relative_attention_max_distance": 128,
  "relative_attention_num_buckets": 32,
  "tie_word_embeddings": false,
  "tokenizer_class": "T5Tokenizer",
  "torch_dtype": "float32",
  "transformers_version": "4.21.2",
  "use_cache": true,
  "vocab_size": 32596
}

[INFO]tokenization_utils_base.py:1723] 2024-05-17 15:49:48,655 >> Didn't find file ./output/spiece.model. We won't load it.
[INFO]tokenization_utils_base.py:1723] 2024-05-17 15:49:48,655 >> Didn't find file ./output/added_tokens.json. We won't load it.
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:49:48,655 >> loading file None
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:49:48,655 >> loading file ./output/tokenizer.json
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:49:48,655 >> loading file None
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:49:48,655 >> loading file ./output/special_tokens_map.json
[INFO]tokenization_utils_base.py:1801] 2024-05-17 15:49:48,655 >> loading file ./output/tokenizer_config.json
[INFO]modeling_utils.py:2039] 2024-05-17 15:49:48,976 >> loading weights file ./output/pytorch_model.bin
[INFO]modeling_utils.py:2435] 2024-05-17 15:49:50,084 >> All model checkpoint weights were used when initializing MT5ForConditionalGeneration.

[INFO]modeling_utils.py:2443] 2024-05-17 15:49:50,084 >> All the weights of MT5ForConditionalGeneration were initialized from the model checkpoint at ./output.
If your task is similar to the task the model of the checkpoint was trained on, you can already use MT5ForConditionalGeneration for predictions without further training.
Running tokenizer on prediction dataset: 0%
Caching processed dataset at /root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecef34c3700d143b31835ec4235bf10109f7/cache-9b3b825ab95ae7e9.arrow
05/17/2024 15:49:50 - INFO - datasets.arrow_dataset - Caching processed dataset at /root/autodl-tmp/dst_code/cache_dir/json/default-45b7308122cad10b/0.0.0/c8d2d9508a2a2067ab02cd118834ecef34c3700d143b31835ec4235bf10109f7/cache-9b3b825ab95ae7e9.arrow
Running tokenizer on prediction dataset: 100%
05/17/2024 15:49:51 - INFO - _main - *** Predict ***
[INFO]trainer.py:2891] 2024-05-17 15:49:51,951 >> ***** Running Prediction *****
[INFO]trainer.py:2893] 2024-05-17 15:49:51,951 >> Num examples = 4643
[INFO]trainer.py:2896] 2024-05-17 15:49:51,951 >> Batch size = 8
100%
**** predict metrics ****
predict_seq_len = 34.5337
predict_joint_acc = 0.7272
predict_loss = 0.1054
predict_runtime = 0:04:56.80
predict_samples = 4643
predict_samples_per_second = 15.643
predict_steps_per_second = 1.958
```

图 16.测试过程图

四、 实验结果

在 output 目录下运行命令行“tensorboard --port 6007 --logdir=./”，可以将结果可视化。

```
C(lab5) root@autodl-container-4fc0119e52-1d7b1e9d: /autodl-tmp/dst_code/output# tensorboard --port 6007 --logdir=./
2024-05-20 14:27:47.766200: I tensorflow/core/util/port.cc:110] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2024-05-20 14:27:47.870244: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2024-05-20 14:27:49.438904: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT
2024-05-20 14:27:51.364172: E tensorflow/compiler/xla/stream_executor/cuda/cuda_driver.cc:268] failed call to cuInit: CUDA_ERROR_NO_DEVICE: no CUDA-capable device is detected
Serving TensorBoard on localhost; to expose to the network, use a proxy or pass --bind all
TensorBoard 2.14.0 at http://localhost:6007/ (Press CTRL-C to quit)
```

图 17. 使用 tensorboard 可视化

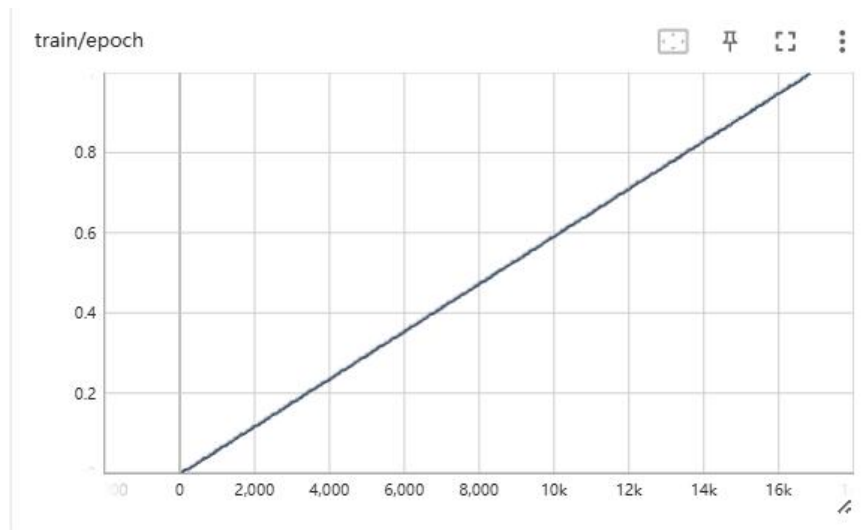


图 18. epoch 随 train 的变化图

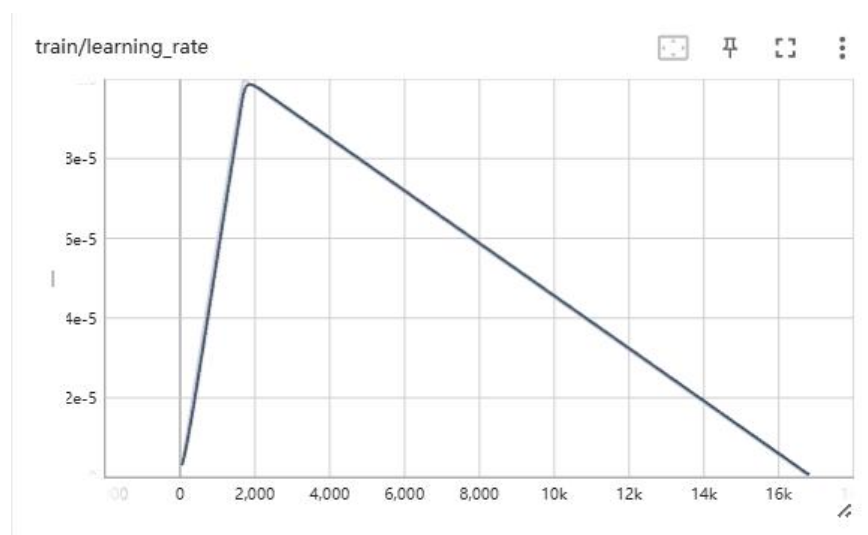


图 19. learning_rate 随 train 的变化图

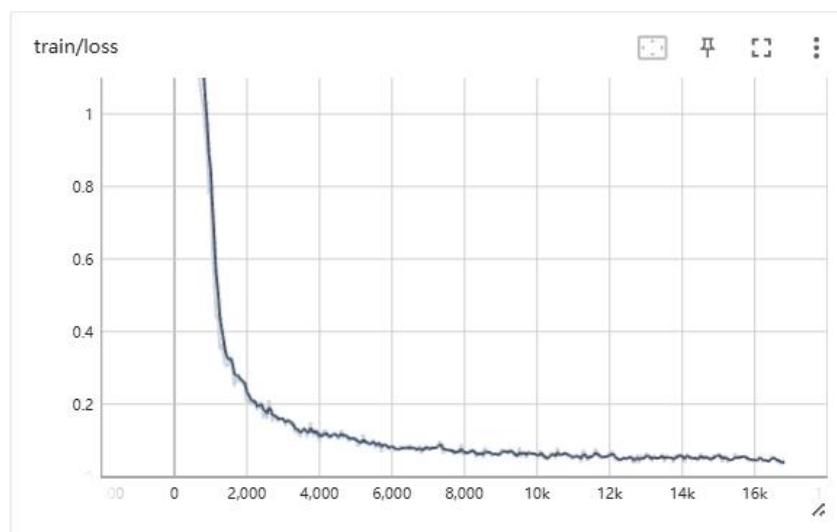


图 20. loss 随 train 的变化图

五、 结果分析

从图 18 可以看出，`epoch` 与 `train` 的训练进度成正比。

从图 19 可以看出，当 `train` 小于等于 2000 时，学习率 `learning_rate` 线性增长，当 `train` 大于 2000 时，学习率 `learning_rate` 线性下降，但下降的速度没有上升时的快。在训练的初始阶段，学习率从较低的值开始线性地增加到一个较大的值。这个过程允许模型在训练初期更快地适应数据，并更快地找到损失函数的下降方向。当学习率达到一定的高度后，开始线性地降低。这是因为随着训练的深入，模型逐渐接近最优解，此时需要更精细的调整来避免在最优解附近震荡或过度拟合。

这种学习率的变化具有一定的优势，在训练的初始阶段，线性增加的学习率可以加快模型的学习速度，使模型更快地适应数据，提高训练效率。通过先增加后减少的学习率，模型可以在训练初期快速找到损失函数的下降方向，然后在后期进行精细的调整，从而得到更好的性能。在训练的后期，线性减少的学习率可以帮助模型在最优解附近进行更精细的调整，避免在最优解附近震荡或过度拟合。

从图 20 可以看出，当 `train` 小于等于 2000 时，损失值 `loss` 近乎直线下降，当 `train` 大于 2000 时，损失值 `loss` 缓慢下降，直至在 0.10 上下浮动。从图中可以得出，模型随着训练的不断推进，预测值逐渐与真实值接近，模型预测结果的准确性，逐渐增加。

六、 心得体会

DST 作为自然语言处理领域中的一项关键技术，对于实现智能对话系统具有重要的应用价值。在进行实验的过程中，我深刻感受到了理论与实践之间的紧密联系，以及不断试错与探索的乐趣。

首先，我对 DST 的基本原理和技术细节有了更深入的了解。在实验中，我学习了如何使用各种算法和模型来追踪对话中的状态信息。这让我认识到，DST 技术的核心在于如何准确地从对话文本中提取出有用的信息，并将其转化为结构化的状态表示。通过研读代码，对于模型的架构组成有了深刻的理解与体会，也激发了我对 DST 的兴趣。

在运行代码的过程中，我对于虚拟环境的配置越来越得心应手，对于实验环境的安装也越来越熟练，虽然在过程中会遇到 `error`，但通过上网查询都一一解决。

数据预处理作为实验的第一部分是及其重要的。在 DST 实验中，数据的质量直接影响到模型的训练效果和性能。在模型训练阶段，使用 `shell` 脚本进行运行，避免了复杂的命令行输入，大大提高模型训练的效率。在实验结果分析阶段，我发现了随着训练的推进，学习率 `learning_rate` 和损失值 `loss` 的变化，通过分析数值变化过程以及原因呢，让我深深感受到，随着模型的不训练，模型的性能逐渐提高，模型预测结果的准确性也大大提高。

在使用 `tensorboard` 进行结果可视化的过程中，面临许多问题，由于我是在服务器进行运行的，因此使用的命令与老师 `ppt` 给的有所不同，但通过上网查阅资料以及和同学交流讨论，最后成功获得可视化结果。在这个过程中，我与

同学们共同讨论问题、分享经验、互相学习。不仅解决了难题，还提高了实验的效率和质量。这让我深刻认识到，在科研工作中，团队合作是不可或缺的。

总之，实现 DST 这个实验让我受益匪浅。我不仅学到了许多新的知识和技能，还提高了自己的实践能力和团队协作能力。我相信这些经验和收获将对我未来的学习和工作产生积极的影响。